Thermal- and Energy-Aware CPU Scheduling with Preference-Conditioned Deep Q-Learning

Gautam (B24CM1024)
GitHub: `https://github.com/gautxmghugharwal/rl-energy-scheduler`

Abstract

We formulate heterogeneous CPU scheduling as a reinforcement-learning problem in which a policy jointly chooses a waiting job, a core type, and a dynamic-voltage-and-frequency-scaling (DVFS) level. A custom simulator models uncertain job-size estimates, energy consumption, temperature-dependent leakage, and thermal throttling. A Double Deep Q-Network (Double DQN), implemented directly in NumPy, is evaluated against Round Robin (RR), Priority, and Shortest Job First (SJF) over six workload scenarios. The learned policy reduces energy and thermal throttling, but typically increases turnaround time.

1 Problem and Environment

The simulator represents a heterogeneous four-core CPU with two high-performance (big) cores and two energy-efficient (LITTLE) cores. A dispatch selects one of three frequency levels, $f \in \{0.5, 0.75, 1.0\}$ relative to maximum frequency. Dynamic power is modelled as $P_{\text{dyn}} = K_c f^3$; leakage increases with temperature, and a big core reaching 85°C is forced to its lowest frequency. The thermal state follows a first-order RC model. Jobs arrive through Poisson or bursty processes and have mixed or heavy-tailed service demands. The scheduler sees noisy burst estimates rather than true remaining work.

2 RL Formulation

State. The observation is a 65-dimensional vector. It contains five features for each core, five global context features (including queue/backlog, utilisation and energy preference), and five features for each of up to eight visible waiting jobs. The visible set combines older, shorter, and higher-priority jobs so that a long queue does not completely hide small or potentially starving jobs.

Action. There are 48 discrete actions: choose one of eight visible job slots, one of two core types (big or LITTLE), and one of three frequency levels ($8 \times 2 \times 3 = 48$). Illegal actions are masked, including empty job slots or a core type with no free core. A dispatched job runs for a four-tick time slice, allowing preemption and migration.

Reward. Each transition incurs the negative of a priority-weighted holding cost plus energy cost:

$$r_t = -\frac{1}{10} \sum_{\tau \in \mathcal{T}_t} \left[\sum_{j \in \mathcal{S}_\tau} \rho_j \left(1 + 0.25 \frac{w_j}{\hat{b}_j + 4} \right) + w_E P_\tau \right],$$

where $\mathcal{T}_t$ is the set of ticks in the transition, $\mathcal{S}_\tau$ the jobs in the system, ρ_j the priority weight, w_j waiting time, $\hat{b}_j$ estimated burst, P_τ total power, and w_E the energy preference. Lower accumulated cost (negative reward) is better; the waiting/size term discourages starvation.

3 Learning Method

The agent uses Double DQN with experience replay, Huber loss, Adam, a soft-updated target network, masked actions, and three-step returns. A shared per-job network scores each visible job from the system context, job features, and relative features; this permutation-equivariant design shares learning across queue positions. Potential-based reward shaping uses a weighted-shortest-processing-time (WSPT) scheduling potential during training. Workload load, arrival pattern, burst distribution, and energy preference are randomised during training. The best checkpoint is selected on held-out validation workloads; test workloads use separate seeds. The simulator, learning code, and baselines are implemented in Python/NumPy.

4 Evaluation and Results

We evaluate at energy weight $w_E = 4$ on six scenarios (light, medium, heavy, bursty, heavy-tail, and overload), with ten unseen seeds per scenario. RR, Priority, and SJF serve as classical baselines. Table 1 reports the lowest-cost classical baseline against DQN; the comparison plot shows all four policies.

Table 1: Evaluation cost; lower is better.

Workload	Best base.	DQN	Winner
Light	1585	1368	DQN
Medium	1654	1527	DQN
Heavy	1996	1956	DQN
Bursty	1670	1572	DQN
Heavy-tail	580	533	DQN
Overload	2312	2415	SJF

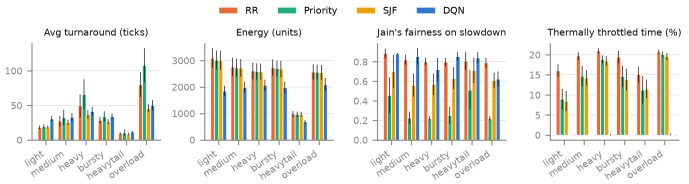


Figure 1: Scheduler comparison across six workloads.

DQN achieves the lowest cost in five of six scenarios, improving on the best baseline by about 2.0–13.7%. On the unseen overload workload, SJF is better (cost 2312 versus 2415), a 4.5% gap. Compared with SJF, DQN uses 17.8–38.5% less energy and experiences only 0.0–0.2% thermal

throttling, versus 8.5–19.6% for SJF. Jain's fairness index is higher for DQN in all six scenarios (0.625–0.882 versus 0.562–0.712 for SJF). The trade-off is turnaround: DQN's mean turnaround is 10.1–64.6% higher than SJF, consistent with the energy-weighted objective.

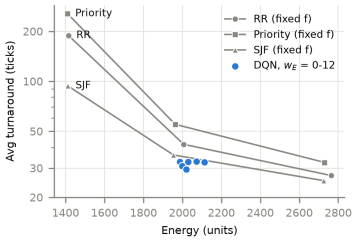


Figure 2: Energy–turnaround trade-off.

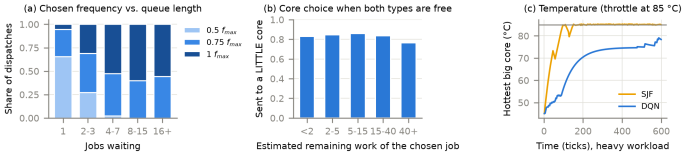


Figure 3: Analysis of learned scheduling choices.

Limitations. The evaluation is simulation-based and uses one training seed; DQN does not win in the unseen overload case, and the energy-preference input appears to affect behaviour only weakly in these runs. Results therefore support an energy/thermal benefit in the tested scenarios, not a claim of universal scheduling superiority.

References

[1] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-learning," *AAAI*, 2016. [2] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," *ICML*, 1999.